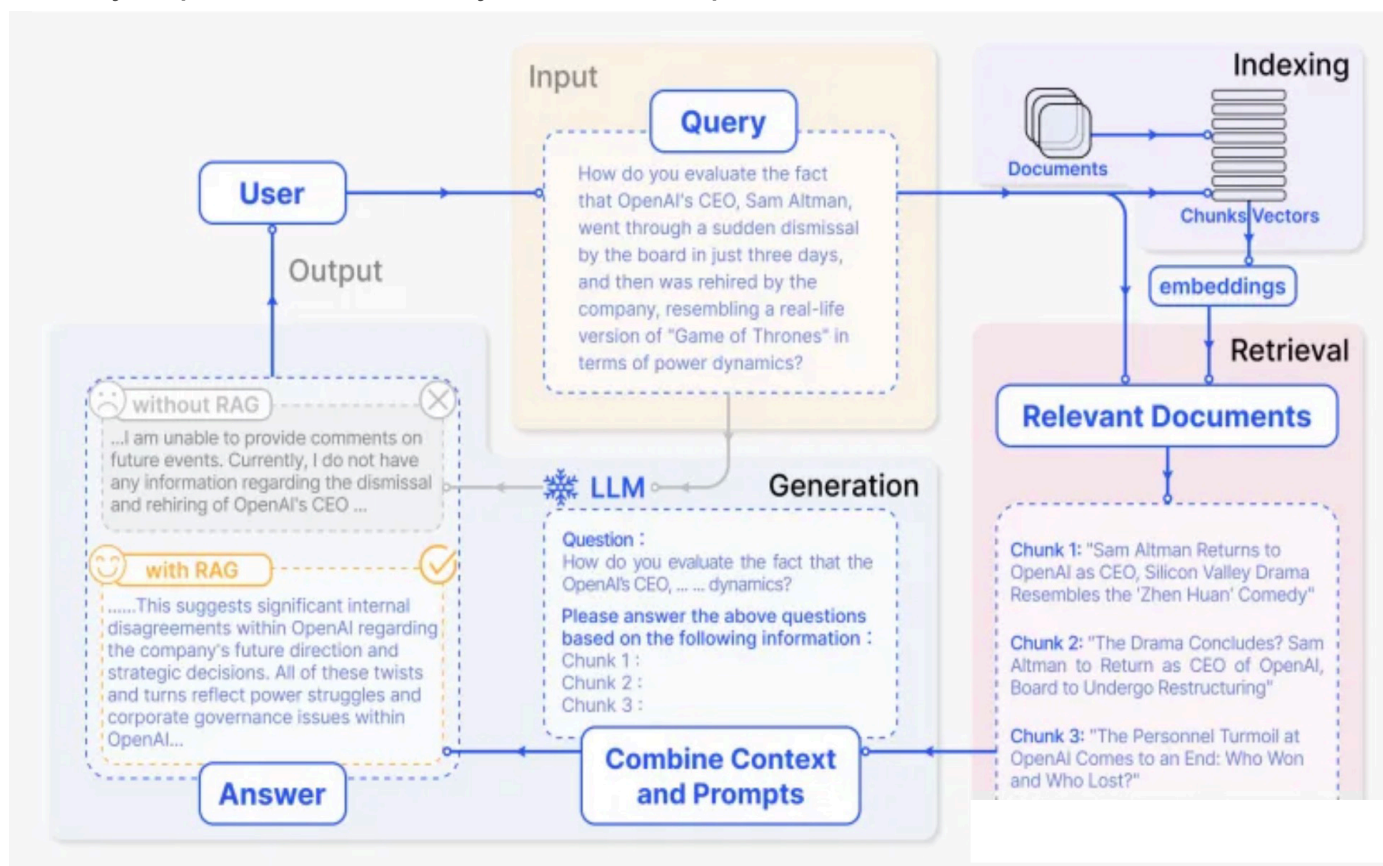


RAG(Retrieval Augmented Generation)

RAG ¹ It is a paradigm for enhancing Large Language Models (LLMs) by integrating external knowledge bases. It employs a collaborative approach, combining information retrieval mechanisms and In-Context Learning (ICL) to improve LLM performance. The RAG workflow consists of three key steps: First, the corpus is divided into discrete chunks, and then a vector index is constructed using an encoder model. Second, RAG identifies and retrieves relevant chunks based on their vector similarity to the query and indexed chunks. Finally, the model synthesizes a response conditioned on the contextual information obtained from the retrieved chunks. These steps constitute the fundamental framework of the RAG process, supporting its information retrieval and context-aware generation capabilities. The key advantage of RAG is that it does not require retraining the LLM for specific tasks; developers can add external knowledge bases to enrich the input and thereby improve the accuracy of model outputs.



The RAG research paradigm has undergone a continuous evolution, mainly divided into three types: Naive RAG, Advanced RAG, and Modular RAG. While RAG is cost-effective and surpasses the performance of native LLMs, it also exhibits some limitations. To address specific drawbacks in Naive RAG, Advanced RAG and Modular RAG were developed.

Naive RAG

Naive RAG follows a traditional process involving indexing, retrieval, and generation.

Indexing. Indexing during the data preparation process is a crucial initial step, involving multiple stages, from data cleaning and extraction to file format conversion and chunking, followed by the transformation into vector representations using embedding models and the creation of an index. The index facilitates similarity comparison during the retrieval stage and stores text segments and their vector embeddings as key-value pairs, enabling efficient and scalable search capabilities.

Retrieval. The system transcodes the user query to generate a vector representation and calculates the similarity score between the query vector and the chunks in the indexed corpus. The system prioritizes retrieving the chunks most similar to the query, which serve as the basis for the augmented context used to respond to the user's request.

Generation. Large language models are employed to synthesize documents based on prompts to formulate a response, allowing them to leverage parametric knowledge or be constrained to the information provided in the retrieved documents. Dialogue history can also be integrated into the prompt for multi-turn conversational interactions. The model's approach to answering questions may vary depending on specific task criteria.

Limitations of Naive RAG. Naive RAG faces challenges in three key areas: retrieval, generation, and augmentation. Low retrieval quality leads to issues such as mismatched chunks, hallucinations, and dropping relevant information, hindering the ability to construct comprehensive responses. Generation quality raises hallucination

challenges, where the model generates answers not grounded in the provided context, alongside issues of irrelevant context and potential toxicity or bias. The augmentation process struggles to effectively combine context from retrieved passages with the current generation task, resulting in incoherent or inconsistent output. Repetition and redundancy are also problems, requiring the determination of the importance and relevance of retrieved passages and the harmonization of different writing styles and tones. There is a risk that the generation model may overly rely on augmented information, potentially leading to output that merely reiterates the retrieved content without providing new value or synthesized information.

Advanced RAG

Advanced RAG addresses the shortcomings of Naive RAG by enhancing targeted retrieval strategies and improving indexing methods. It implements pre-retrieval and post-retrieval strategies and employs techniques such as sliding windows, fine-grained segmentation, and metadata to refine indexing. Furthermore, various approaches like ILIN are introduced to optimize the retrieval process.

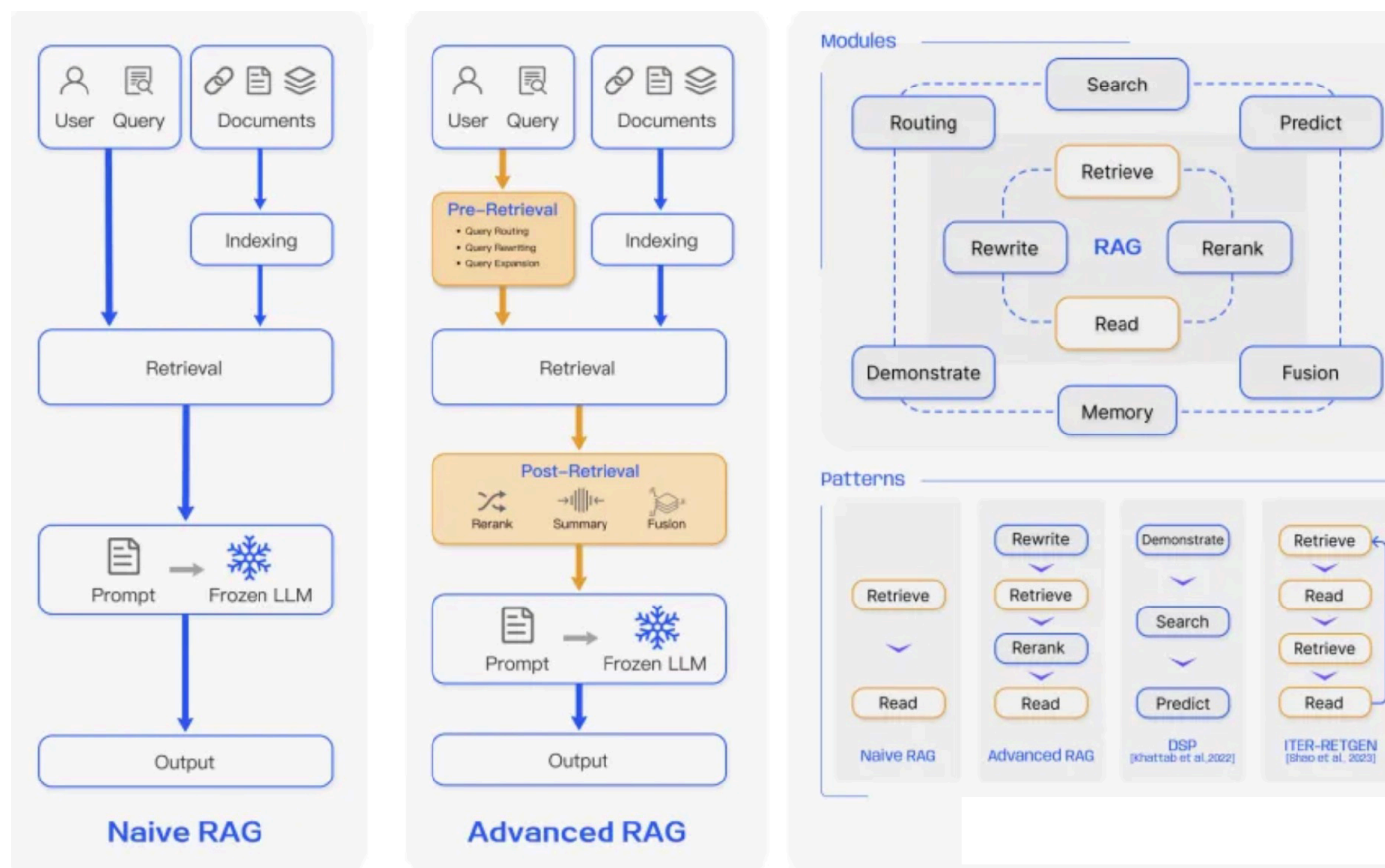
Pre-retrieval Process. Optimizing data indexing can improve the quality of indexed content, thereby enhancing the performance of the RAG system. This involves five main strategies: augmenting data granularity, optimizing the index structure, adding metadata, alignment optimization, and hybrid retrieval. Augmenting data granularity aims to increase the level of text standardization and contextual richness. Optimizing the index structure includes adjusting chunk size and leveraging graph data indexes to integrate information. Adding metadata information is used for filtering and integrating metadata to improve retrieval efficiency. Alignment optimization addresses alignment issues and discrepancies between documents.

Retrieval Stage. In the retrieval stage, appropriate context is determined by calculating the similarity between the query and the chunks, with embedding models being central. Fine-tuning embedding models can enhance retrieval relevance in specific domain contexts, and dynamic

embeddings adapt based on the context used. Transformer models like BERT can capture contextual understanding but might be less context-sensitive than the latest full-sized language models.

Post-retrieval Process. After the valuable context retrieved from the database is merged with the query, addressing challenges posed by context window limitations, re-ranking and prompt compression are key strategies. Re-ranking methods include prioritizing based on document diversity, alternately placing the best documents at the beginning and end of the context window, and recomputing the semantic similarity between relevant text and the query. Prompt compression methods involve leveraging small language models to calculate the mutual information or perplexity of prompts, as well as designing summarization techniques to enhance the LLM's perception of key information.

Modular RAG



The Modular RAG structure is a novel RAG framework characterized by greater flexibility and adaptability, capable of integrating various methods to enhance functional modules and address specific problems. The Modular RAG paradigm is gradually becoming the norm, allowing for serial pipelines or end-to-end training approaches across multiple modules. Advanced RAG is a specialized form of Modular RAG, while Naive RAG is a special case of Advanced RAG. The relationship between these three paradigms is one of inheritance and development.

New Module: Search Module. This module is customized for specific scenarios and incorporates direct searches of additional corpora, with integration achieved through LLM-generated code, query languages, and other customized tools. Data sources include search engines, text data, tabular data, and knowledge graphs.

Memory Module. This module leverages the LLM's memory capabilities to guide retrieval, identifying memories most similar to the current input and iteratively creating an unbounded memory pool through a retrieval-augmented generator, making the text more consistent with the data distribution during inference.

Fusion Approach. This approach enhances traditional search systems by employing multi-query methods, using LLMs to expand user queries into multiple distinct perspectives, uncovering deeper transformative knowledge. The fusion process involves parallel vector searches of original and expanded queries, intelligent re-ranking to optimize results, and pairing the best results with new queries.

RAG System Retrieval Process. This process utilizes multiple sources, used alternately or merged depending on the situation. Query routing determines the subsequent actions for user queries, including summarization, searching specific databases, or merging different paths into a single response. Query routers select the appropriate data storage for the query, with decisions being predefined and executed through LLM calls. The prediction module uses LLMs to generate necessary content, addressing redundancy

and noise in the retrieved content, making the generated content more likely to contain relevant information.

Task Adapter. This module enables RAG to adapt to various downstream tasks. UPRISE automatically retrieves prompts from pre-built data pools, supporting zero-shot task input and enhancing cross-task and model universality. PROMPTAGATOR uses LLMs as few-shot query generators, creating task-specific retrievers, leveraging a small number of examples to develop end-to-end task-specific retrievers.

New Mode

Modular RAG exhibits high adaptability, allowing for the replacement or rearrangement of modules within the RAG process to suit specific problem contexts. Current research primarily explores two organizational paradigms: one involving the addition or replacement of modules, and the other focusing on adjusting the organizational flow between modules. This flexibility enables the customization of the RAG process as needed, effectively handling a wide range of tasks.

Optimizing the RAG Pipeline

Optimizing the RAG pipeline to enhance information efficiency and quality involves achieving a balance between retrieval efficiency and the depth of contextual information by integrating various search techniques, improving retrieval steps, incorporating cognitive backtracking, implementing versatile query strategies, and leveraging embedding similarity. Methods such as exploring hybrid search, recursive retrieval and query engines, backward prompting, sub-queries, and hypothetical document embeddings are used to optimize the RAG pipeline. These methods help in generating responses to backward prompts and the final answer generation process. However, this approach may not always yield satisfactory results, especially when the language model is

unfamiliar with the subject, potentially leading to more instances of errors.

GraphRAG [2](#)

GraphRAG is a structured, hierarchical approach to Retrieval Augmented Generation (RAG), as opposed to naive semantic-search approaches using plain text snippets. The GraphRAG process involves extracting a knowledge graph out of raw text, building a community hierarchy, generating summaries for these communities, and then leveraging these structures when perform RAG-based tasks.

GraphRAG vs Baseline RAG

Retrieval-Augmented Generation (RAG) is a technique to improve LLM outputs using real-world information. This technique is an important part of most LLM-based tools and the majority of RAG approaches use vector similarity as the search technique, which we call Baseline RAG. GraphRAG uses knowledge graphs to provide substantial improvements in question-and-answer performance when reasoning about complex information. RAG techniques have shown promise in helping LLMs to reason about private datasets - data that the LLM is not trained on and has never seen before, such as an enterprise's proprietary research, business documents, or communications. Baseline RAG was created to help solve this problem, but we observe situations where baseline RAG performs very poorly. For example: Baseline RAG struggles to connect the dots. This happens when answering a question requires

traversing disparate pieces of information through their shared attributes in order to provide new synthesized insights.

Baseline RAG performs poorly when being asked to holistically understand summarized semantic concepts over large data collections or even singular large documents.

To address this, the tech community is working to develop methods that extend and enhance RAG.

Microsoft Research's new approach, GraphRAG, creates a knowledge graph based on an input corpus.

This graph, along with community summaries and graph machine learning outputs, are used to augment prompts at query time. GraphRAG shows substantial improvement in answering the two classes of questions described above, demonstrating intelligence or mastery that outperforms other approaches previously applied to private datasets.

The GraphRAG Process 

GraphRAG builds upon our prior research and tooling using graph machine learning. The basic steps of the GraphRAG process are as follows:

Index

Slice up an input corpus into a series of TextUnits, which act as analyzable units for the rest of the process, and provide fine-grained references in our outputs.

Extract all entities, relationships, and key claims from the TextUnits.

Perform a hierarchical clustering of the graph using the Leiden technique. To see this visually, check out

Figure 1 above. Each circle is an entity (e.g., a person, place, or organization), with the size representing the degree of the entity, and the color representing its community. Generate summaries of each community and its constituents from the bottom-up. This aids in holistic understanding of the dataset.

Query

At query time, these structures are used to provide materials for the LLM context window when

answering a question. The primary query modes are:

Global Search for reasoning about holistic questions about the corpus by leveraging the community summaries.

Local Search for reasoning about specific entities by fanning-out to their neighbors and associated concepts.

DRIFT Search for reasoning about specific entities by fanning-out to their neighbors and associated concepts, but with the added context of community information.

Prompt Tuning Using GraphRAG with your data out of the box may not yield the best possible results. We strongly recommend to fine-tune your prompts following the Prompt Tuning Guide in our documentation.